

RESEARCH ARTICLE OPEN ACCESS

A Multirepresentation Stacked Ensemble for AI-Generated Text Detection

Md. Siam Ansary 

Department of Computer Science and Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh

Correspondence: Md. Siam Ansary (siamansary.cse@gmail.com; siamansary.cse@aust.edu)**Received:** 14 February 2026 | **Revised:** 26 March 2026 | **Accepted:** 10 April 2026**Academic Editor:** Richard Murray**Keywords:** AI-generated text detection | NLP | text classification

ABSTRACT

The rapid advancement of large language models (LLMs) has led to a substantial increase in AI-generated text, raising concerns regarding academic integrity, content authenticity, and misinformation. Detecting such text has become increasingly challenging due to improved fluency, paraphrasing capabilities, and domain variability. This paper proposes a multirepresentation stacked ensemble framework for AI-generated text detection that integrates heterogeneous feature spaces, including transformer-based contextual embeddings, contrastive semantic representations, handcrafted linguistic features, and LLM-derived meta-features. A mutual information-based feature selection strategy is employed to reduce redundancy and enhance generalization. The selected features are processed through a multilevel ensemble consisting of diverse base learners and a logistic regression meta-classifier. The proposed approach is evaluated on two publicly available benchmark datasets. Experimental results demonstrate that the model achieves an accuracy of 98.74% and an AUC of 0.997 on Dataset 01, and an accuracy of 97.92% with an AUC of 0.994 on Dataset 02. Comparative analysis with multiple baseline models and an ablation study confirm that the integration of heterogeneous representations significantly improves detection performance over single-model approaches. These findings indicate that combining semantic, statistical, and meta-level features within a unified ensemble framework provides a robust and generalizable solution for AI-generated text detection.

1 | Introduction

The rapid advancement of large language models (LLMs) has fundamentally transformed natural language generation, enabling machines to produce text that is highly fluent, coherent, and often indistinguishable from human writing. Systems such as ChatGPT and similar generative models are increasingly used across academic, industrial, and social domains. While these technologies offer substantial benefits, they also introduce critical challenges related to academic integrity, content authenticity, misinformation, and automated content manipulation.

As LLM-generated text becomes more sophisticated, detecting machine-generated content has emerged as a significant research

problem. Traditional detection approaches, which rely on lexical statistics, n-gram distributions, or stylometric features, often fail to generalize across domains and generation styles. Moreover, modern LLMs can paraphrase, refine, and adapt text stylistically, thereby reducing detectable artifacts and making the distinction between human-authored and AI-generated text increasingly subtle.

Recent research has explored a wide range of detection strategies, including fine-tuned transformer models, contrastive representation learning, stylometric feature engineering, and ensemble-based methods. Transformer-based approaches leverage contextual embeddings to capture deep semantic patterns, but they are often sensitive to domain shifts and adversarial rewriting. On

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Copyright © 2026 Md. Siam Ansary. *International Journal of Intelligent Systems* published by John Wiley & Sons Ltd.

the other hand, handcrafted linguistic features provide interpretable and robust statistical signals but lack expressive power when used in isolation. Similarly, contrastive learning methods capture semantic similarity structures but may overlook stylistic nuances. These limitations suggest that relying on a single representation or modeling paradigm is insufficient for robust AI-generated text detection.

Motivated by these observations, this paper proposes a multi-representation stacked ensemble framework that integrates complementary feature spaces within a unified architecture. Specifically, the proposed approach combines (i) transformer-based contextual embeddings, (ii) contrastive semantic representations, (iii) handcrafted linguistic features, and (iv) meta-features derived from a lightweight LLM. These heterogeneous representations are fused and processed using a multilevel stacked ensemble consisting of diverse base learners and a meta-classifier. In addition, a mutual information-based feature selection strategy is employed to reduce redundancy and improve generalization.

To validate the effectiveness of the proposed framework, extensive experiments are conducted on two publicly available benchmark datasets. The evaluation includes not only standard performance metrics such as accuracy, F1 score, and area under the ROC curve (AUC) but also comparative analysis with multiple baseline models and a comprehensive ablation study. These analyses provide insights into the contribution of each feature group and the overall robustness of the model.

The main contributions of this work can be summarized as follows:

- We propose a novel multirepresentation framework that integrates semantic, statistical, and meta-level features for AI-generated text detection.
- We design a stacked ensemble architecture that effectively combines diverse learning paradigms to improve classification performance.
- We introduce an LLM-based meta-feature extraction mechanism that captures high-level stylistic and reasoning signals.
- We conduct extensive experiments, including baseline comparisons and ablation studies, to demonstrate the effectiveness and robustness of the proposed approach.

Overall, this work contributes to the development of more reliable and generalizable methods for detecting AI-generated text in increasingly complex and evolving generation environments.

2 | Literature Review

We have studied the existing research studies on AI-generated text detection to gain insights that can be very helpful.

Abburi et al. [1] conducted AI-generated text detection using an ensemble method. Several datasets were evaluated. Under the employed method, every input was fed through five pretrained models, which were DeBERTa large, RoBERTa with XNLI, RoBERTa large, RoBERTa base OpenAI detector, and XLM-RoBERTa NLI. To maximize the benefits of the methods, separately generated class probabilities of each model were

combined and given as an input to train a voting classifier, which included logistic regression, random forest, Gaussian Naive Bayes, and support vector machine. The maximum accuracy of the experiment was 78.4%.

Shah et al. [2] employed a combining method that has stylistic characteristics in the detection of AGTs. Various facets of lexical features, readability scores, diversity, and richness of the vocabulary were taken into consideration. The researchers made custom datasets for the research. The initial classification in the experiment was logistic regression, decision tree, random forest, support vector machine, and gradient boosting. The final scores of the two best classifiers were combined with two methods of xAI analysis, namely, SHAP and LIME. The hybrid solution presented strong results.

Yan et al. [3] prepared a bespoke dataset and were able to use RoBERTa and SVM to detect AGT. The RoBERTa model was fine-tuned, and the SVM model was trained on a few features created by the in-house scoring engine. RoBERTa was able to reach 99.75%, and SVM reached 96%.

Mitchell et al. used probability curvature to detect machine-generated texts [4]. The researchers conjectured that text generated by LLM is of a negative form. They confirmed that small rewrites of the texts created by models are less likely to be generated under the model compared with the original text and vice versa in the case of human-written texts.

In their work on AGT detection with pretrained models, Yadagiri et al. concentrated on various linguistic features [5]. The scholars developed a dataset containing about forty thousand queries and ChatGPT and human answers. CNN-BiLSTM, RNN, GPT-2, and RoBERTa were the models in use. The strongest of the models had an accuracy of 99.73, which was RoBERTa.

Zhang et al. [6] took a hybrid method of AI-generated text detection. In the hybrid model, a Bayesian classifier was used together with TF-IDF, SGD classifier, LightGBM classifier, CatBoost classifier, and BPE tokenization. Subsequently, 12 instances of DeBERTa v3 large models were used in this ensemble pipeline. The experimental method achieved the highest ROC-AUC of 97.5%.

Qazi et al. [7] created a new dataset and suggested a model of GPT-generated text detection in English. The authors developed the GPT Reddit Dataset (GRiD), which contains 6513 samples, of which 1368 were created using the GPT-3.5-turbo model and the rest were created by humans. The human-written texts were scraped on Reddit through the PRAW Python and the posts that were written prior to the official launch of the ChatGPT web application. The scientists suggested a new approach according to which they used tensor decomposition as a tool for recognizing fundamental patterns of data GPTEN. BERT, SVM, and random forest, as well as the proposed model, were utilized by the scientists. BERT was more effective than the other three methods.

Gryka and colleagues [8] in their research presented a set of machine learning classifiers to compare them in relation to detecting artificially generated email content. All were considered: the decision tree, random forest (using a hundred estimators), multilayer perceptron, k-nearest neighbors ($k = 5$), support vector machine (with radial basis function kernel), AdaBoost (50 decision trees with learning rate = 1), Gaussian

Naive Bayes, and quadratic discriminant analysis. The corpus was composed of over ten thousand English and Polish messages. Here, the random forest algorithm had the most optimal performance with the F1 score of 98.8% on the English subset and 92.1% on the Polish subset.

Zhou and Wang [9] carried out an experiment on AI-generated prose detection in various fields. They combined multiple existing corpora to form a composite dataset to be experimented with. Their contribution in terms of methodology was an adapted version of RoBERTa, called RoBERTa-Ranker, in which a loss margin ranking function was added, and a mean pooling layer was added to the encoder output, which further enriched the sentence representation before classification.

Guo et al. [10] suggested a multilevel contrastive learning and multitask objective end-to-end pipeline to detect AI-generated text. First, training samples were pre-encoded, and salient features were derived, and similarities between the encoded representations and a collection of feature vectors were compared to every textual instance. The last predictive step used a k-nearest neighbor algorithm to give classifications.

Wu and colleagues [11] proposed a set of LLM-generated content detectors. Their sample was made up of human-written passages filtered down to existing corpora and publicly available sites compared to the output of GPT-3.5-turbo, PaLM-2-bison, Claude-instant, and Llama-2-70B. This balanced mix was tested using systematic experiments to determine the performance of various classifiers.

Guo and coworkers [12] later had a deterministic view of causal language models. They maintained that such models, when provided with data written by humans, store a lot of contextual token data at the expense of making predictions about the next token. Their analysis model used bidirectional cross-entropy of generated logits and the ground-truth forward token, and logits and the previous backward token.

Wang et al. [13] used the GBERT architecture on the problem of identifying AI-generated text. They had a proprietary dataset of 670 human-generated samples and 708 algorithmically generated samples. The neural network had average training and test accuracies of 98.1 and 97.71, respectively.

Yu et al. [14] were interested in the creation of a benchmark to recognize the abstracts created by ChatGPT. This amounted to 15,395 human-created abstracts and 35,304 machine-created ones, obtained via IEEE Xplore based on defined keywords. The latter were divided into three categories, namely, generation (completely algorithmic abstracts), Polish (text modeled and refined by a human), and mix (hybrid model-polished and human text). A variety of BERT variants were tested on this dataset; the highest AUCs were achieved on the Polish partition (99.56), generation (100), and mix (87.83).

Pudasaini et al. [15] compared ArguGPT, RADAR (Robust AI Text Detector through Adversarial Learning), and OpenAI Detector in the framework of AI-generated text detection. They were evaluated based on the M4 and HC3 datasets.

According to the assumptions expressed by McGovern and co-authors [16], it is possible to perceive AI-generated textual material with relatively simple classifiers, as long as informative features are extracted. They used a GradientBoost classifier and

three separate sets of features, including word n-grams, character n-grams, and part-of-speech n-grams. In several datasets, their algorithm gave encouraging detection rates.

Varadarajan et al. [17] reasoned that the text produced by LLM has poor human-like authenticity and richness. They laid stress on the significance of human qualities like demographics, personality, empathy, and the behavior linguistic characteristics. They used a wide variety of datasets, implementing an unsupervised classification pipeline, which used spectral clustering using a radial basis kernel.

Valdez-Valenzuela et al. [18] presented a graph-based classification system that is taxonomy-aware and a combination of text representation and graph neural networks for the identification of AI-generated content. They explored different methods of initializing the features of nodes and formed a co-occurrence graph. Language models that have been pretrained (BERT-base-uncased or RoBERTa-base) were fine-tuned. The resulting graph embeddings were subsequently passed through a classifier that consisted of a dense neural network layer. The Autexification-2023 dataset was used as an experimental foundation.

Abbas [19] investigated the identification of AI-generated text using a zero-shot prompting approach with the help of Sentence-BERT embeddings. Also, a graph convolutional network was incorporated, resulting in increased accuracy. All empirical assessments were based on the PAN-AP-2019 data.

In a similar work, the authors [20] created a dataset of 509 human- and AI-generated question-answer pairs and tested them using several machine-based and deep learning-based classification models. Their approach that uses RoBERTa demonstrated high performance as compared to the other architectures that were tested.

Alhijawi and colleagues [21] selected a collection of 3000 records, which were in turn divided into human-written, AI-generated, and mixed content: 1000 records each. They tested a hybrid architecture of multilayer perceptron and convolutional neural network elements and finally achieved a top-ranking accuracy of 0.876 in the experiment.

3 | The Dataset

We have used two publicly available datasets from the Kaggle platform. The datasets have been experimented with separately.

- Dataset 01: DAIGT V2 dataset [22] is the first dataset that we have done our research with. It has been uploaded by Darek Kłeczek [23] for AI-generated text detection. The dataset has 44,868 text records, and 27,371 records are human-written, while the rest are AI-generated. The AI-generated texts are labeled as 1, while texts written by humans are labeled as 0. The dataset is available in CSV file format. The percentage of category-wise records in the dataset is illustrated in Figure 1.
- Dataset 02: LLM-Detect AI-Generated Text dataset [24] is the second dataset that we have used. It was uploaded by Sunil Thite [25]. 0 is used to indicate the human-written essays, whereas 1 is for the AI-generated essays. A total of 17,508 essays are written by humans, and 11,637 essays are AI-generated. The percentage of category-wise records in the dataset is illustrated in Figure 2.

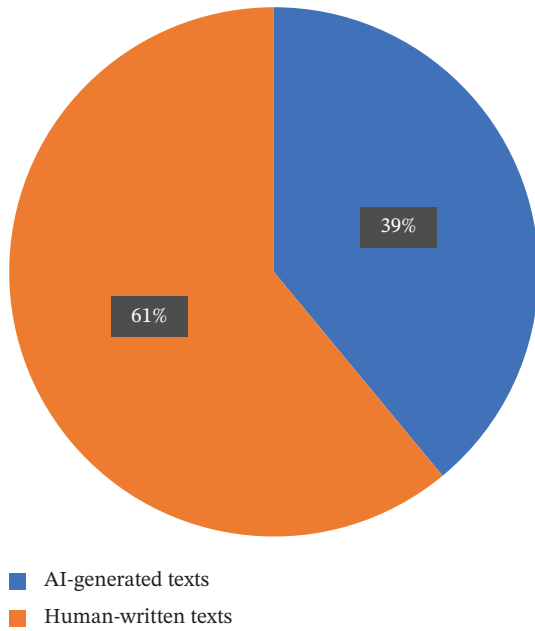


FIGURE 1 | Percentage of category-wise records in the DAIGT V2 dataset.

4 | Methodology

4.1 | Problem Formulation and Research Context

Let the dataset be defined as

$$D = \{(x_i, y_i)\}_{i=1}^N, \quad (1)$$

where x_i represents a text document and $y_i \in \{0, 1\}$ denotes its class label, with $y_i = 1$ indicating AI-generated text and $y_i = 0$ indicating human-authored text.

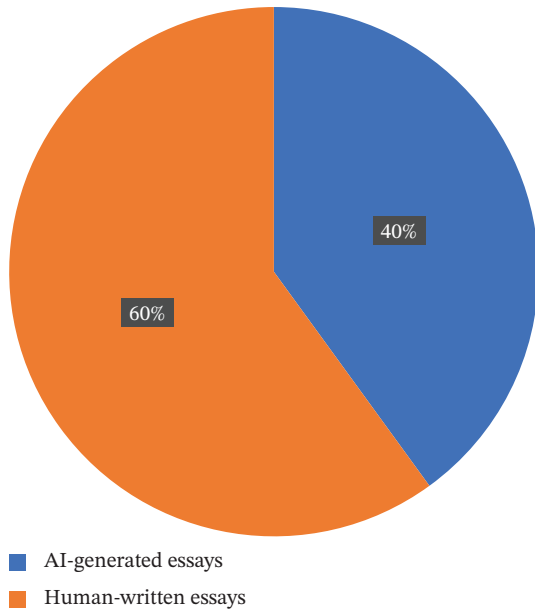


FIGURE 2 | Percentage of category-wise records in the LLM-Detect AI-Generated Text dataset.

The learning objective is to construct a classifier,

$$f: \mathcal{X} \rightarrow \{0, 1\}, \quad (2)$$

that minimizes the expected empirical risk:

$$\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim D} [\mathcal{L}(f(x), y)], \quad (3)$$

where $\mathcal{L}(\cdot)$ denotes the classification loss.

AI-generated text detection differs fundamentally from conventional text classification due to distributional drift, stylistic homogenization, semantic overlap with human writing, and adversarial paraphrasing capabilities of modern LLMs. These challenges necessitate a hybrid, multirepresentational learning strategy.

4.2 | Overall Framework and Design Principles

The proposed framework models the prediction function as

$$f(x) = g(\Phi_T(x), \Phi_C(x), \Phi_H(x), \Phi_L(x)), \quad (4)$$

where $\Phi_T(x)$ denotes transformer-based contextual embeddings, $\Phi_C(x)$ represents contrastive semantic representations, $\Phi_H(x)$ captures handcrafted linguistic statistics, $\Phi_L(x)$ denotes LLM-derived meta-features, and $g(\cdot)$ is a multilevel stacked ensemble classifier.

The framework is guided by four core principles: (i) representation diversity, (ii) model-agnostic robustness, (iii) semantic density awareness, and (iv) hierarchical decision fusion.

4.3 | Dataset Preparation and Stratified Sampling

To mitigate class imbalance effects, stratified sampling is employed. For each class $k \in \{0, 1\}$,

$$\frac{|D_{\text{train}}^k|}{|D_{\text{train}}|} \approx \frac{|D_{\text{test}}^k|}{|D_{\text{test}}|}. \quad (5)$$

The dataset is split into training (70%), validation (15%), and test (15%) subsets. This strategy ensures unbiased gradient estimation, stable ensemble convergence, and fair generalization assessment.

4.4 | Transformer-Based Contextual Embeddings

4.4.1 | Tokenization and Encoding

Each document x is tokenized into a sequence $(t_1, t_2, \dots, t_L)$. A pretrained transformer encoder produces contextual token representations:

$$\mathbf{H} = \text{Encoder}(t_1, \dots, t_L), \quad \mathbf{H} \in \mathbb{R}^{L \times d}. \quad (6)$$

Three encoders are used: BERT-base, RoBERTa-base, and DistilBERT-base. All encoder parameters are frozen to avoid dataset-specific overfitting and preserve general linguistic representations.

4.4.2 | Pooling Strategy and Hyperparameters

Sentence-level embeddings are obtained using mean pooling:

$$\mathbf{e} = \frac{1}{L} \sum_{j=1}^L \mathbf{H}_j. \quad (7)$$

Mean pooling is preferred over CLS token representations due to its stability for long documents and robustness to tokenization artifacts.

The transformer embedding dimension is fixed at $d = 768$. Maximum sequence length is controlled via tokenizer truncation to ensure computational feasibility.

4.4.3 | Multienncoder Fusion

The transformer feature vector is defined as

$$\Phi_T(x) = [\mathbf{e}_{\text{bert}}, \mathbf{e}_{\text{roberta}}, \mathbf{e}_{\text{distilbert}}]. \quad (8)$$

Each encoder contributes complementary inductive biases: syntactic sensitivity (BERT), semantic richness (RoBERTa), and stylistic abstraction (DistilBERT).

4.5 | Contrastive Semantic Representation Learning

A sentence-transformer trained under a contrastive objective maps each document to

$$\mathbf{c}_i = \Phi_C(x_i) \in \mathbb{R}^{384}. \quad (9)$$

The encoder is optimized using contrastive loss:

$$\mathcal{L}_{CL} = -\log \frac{\exp(\cos(\mathbf{c}_i, \mathbf{c}_i^+)/\tau)}{\sum_j \exp(\cos(\mathbf{c}_i, \mathbf{c}_j)/\tau)}, \quad (10)$$

where cosine similarity is used and τ controls embedding sharpness.

The embedding dimension is deliberately kept compact to emphasize semantic density rather than lexical detail.

4.6 | Handcrafted Linguistic Feature Engineering

Handcrafted features are defined as

$$\Phi_H(x) = [f_1(x), f_2(x), \dots, f_6(x)]. \quad (11)$$

These include token count, character count, average token length, lexical diversity, readability score, and Shannon entropy:

$$H(x) = - \sum_{w \in V(x)} p(w) \log p(w). \quad (12)$$

These features are intentionally low-dimensional and model-agnostic, capturing statistical regularities that persist across LLM generations.

4.7 | LLM-Based Meta-Feature Generation

An instruction-tuned lightweight LLM is used to generate

$$\Phi_L(x) = [a, f, p], \quad (13)$$

where a denotes AI-likelihood, f denotes formality, and p denotes predictability.

Decoding hyperparameters are fixed as temperature $T = 0$ and maximum new tokens set to 20, ensuring deterministic and reproducible outputs.

4.7.1 | Model Specification

The LLM used for meta-feature extraction is LLaMA-2-7B-Chat (Meta AI), an instruction-tuned conversational model with approximately 7 billion parameters. This model is selected due to its strong balance between reasoning capability and computational efficiency, making it suitable for large-scale feature extraction tasks.

The model is used in inference-only mode, and no fine-tuning is performed. All meta-features are generated using a fixed and deterministic prompting strategy to ensure reproducibility.

4.7.2 | Prompt Design

A structured prompt is designed to elicit consistent and quantifiable responses from the LLM. The prompt explicitly instructs the model to evaluate the input text along three dimensions:

- AI-likelihood (a): Estimated probability that the text is AI-generated
- Formality (f): Degree of linguistic formality
- Predictability (p): Degree of repetitiveness and structural predictability

The exact prompt template used is as follows.

You are an expert linguistic analyst. Given the following text, provide three scores between 0 and 1:

1. AI-likelihood: How likely is this text generated by an AI model?
2. Formality: How formal is the writing style?
3. Predictability: How predictable or repetitive is the text?

Return your answer strictly in the format:

AI: <value>, Formality: <value>, Predictability: <value>

This structured format ensures that the outputs can be reliably parsed into numerical features.

4.7.3 | Decoding Configuration

To guarantee deterministic and reproducible outputs, the following decoding parameters are fixed:

- Temperature (T): 0.0
- Top-p: 1.0
- Maximum new tokens: 20
- Sampling: disabled (greedy decoding)

Setting $T = 0$ eliminates randomness in generation, ensuring that identical inputs always yield identical outputs.

4.7.4 | Output Parsing and Normalization

The LLM outputs are postprocessed using a rule-based parser to extract numerical values corresponding to the three meta-features. Any malformed outputs are filtered and reprocessed to ensure consistency.

The extracted values are normalized to the range (0, 1) and represented as

$$\Phi_L(x) = [a(x), f(x), p(x)], \quad (14)$$

where

- $a(x)$ denotes AI-likelihood
- $f(x)$ denotes formality
- $p(x)$ denotes predictability

4.7.5 | Design Rationale

The inclusion of LLM-derived meta-features is motivated by the hypothesis that modern language models encode implicit knowledge about stylistic and generative patterns. By querying an LLM directly, we extract higher-level signals that complement traditional feature representations.

Unlike transformer embeddings, which encode token-level context, or handcrafted features, which capture statistical properties, LLM-based meta-features provide a form of *meta-reasoning* about the text.

4.7.6 | Computational Considerations

To control computational cost, the LLM is used in a lightweight inference configuration without gradient computation. Batch processing is applied where possible, and feature extraction is performed offline prior to model training.

4.7.7 | Reproducibility and Transparency

All components of the meta-feature extraction process—including model selection, prompt design, decoding parameters, and parsing rules—are explicitly defined to ensure full reproducibility. This addresses a common limitation in LLM-based pipelines, where unspecified prompts or model configurations hinder experimental validation.

4.7.8 | Limitations

It is important to note that LLM-based meta-features may introduce bias depending on the underlying model's training data and alignment objectives. Future work may explore alternative lightweight models or prompt calibration strategies to further improve robustness.

4.8 | Feature Fusion and Normalization

All feature groups are concatenated:

$$\mathbf{z}_i = [\Phi_T(x_i), \Phi_C(x_i), \Phi_H(x_i), \Phi_L(x_i)]. \quad (15)$$

Z score normalization is applied:

$$\mathbf{z}'_i = \frac{\mathbf{z}_i - \mu}{\sigma}, \quad (16)$$

where μ and σ are computed on the training set.

4.9 | Feature Selection via Mutual Information

Mutual information is computed as

$$MI(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}. \quad (17)$$

The top $K = 500$ features with the highest MI scores are retained to reduce redundancy and improve generalization.

4.10 | Multilevel Stacked Ensemble Classification

Each base classifier learns

$$h_k(\mathbf{z}') \rightarrow [0, 1]. \quad (18)$$

4.10.1 | Base Learners and Hyperparameters

The ensemble framework incorporates a diverse set of base learners, each configured with carefully selected hyperparameters to balance model complexity, generalization capability, and computational efficiency. The detailed configurations are summarized as follows:

- **XGBoost:** The model was trained with $n_{\text{estimators}} = 300$ boosting rounds, a maximum tree depth of 10, and a learning rate of 0.03. To reduce overfitting, subsampling and column sampling rates were set to 0.9 and 0.8, respectively. Regularization was enforced using L1 penalty ($\text{reg}_{\alpha} = 0.5$) and L2 penalty ($\text{reg}_{\lambda} = 1.0$).
- **LightGBM:** The LightGBM classifier employed 300 estimators with a maximum depth of 9 and 100 leaves per tree. A learning rate of 0.03 was used, along with subsampling (0.9) and feature fraction (0.8) to enhance generalization.
- **Random Forest and Extra Trees:** Both models were trained using 300 trees with a maximum depth of 20 and a minimum of 3 samples per leaf. Class imbalance was handled by applying balanced class weights.
- **Gradient Boosting:** This model utilized 250 estimators, a learning rate of 0.05, and a maximum tree depth of 5, providing a controlled trade-off between bias and variance.
- **Support Vector Machine (RBF Kernel):** The SVM classifier was configured with a regularization parameter of $C = 5$. Probability estimation was enabled, and balanced class weights were applied to address class imbalance.
- **k-Nearest Neighbors:** The k-NN model used $k = 5$ nearest neighbors with distance-based weighting to give higher influence to closer instances.
- **CatBoost:** The CatBoost classifier was trained using the default tree depth with a *logloss* objective function. Model verbosity was disabled to streamline training output.

4.10.2 | Meta-Classfier

A logistic regression model with L2 regularization combines base predictions:

$$\hat{y} = \sigma \left(\sum_k w_k h_k(\mathbf{z}') \right). \quad (19)$$

Stratified fivefold cross-validation is used during stacking to prevent information leakage.

4.11 | Retrieval-Augmented Refinement

Contrastive embeddings are indexed using FAISS with an exact FlatL2 index. For each test instance, the $k = 3$ nearest neighbors are retrieved:

$$\mathcal{N}(x) = \underset{i}{\operatorname{argmin}} \|\mathbf{c}_x - \mathbf{c}_i\|. \quad (20)$$

Retrieved samples provide contextual grounding, reducing overconfident predictions on ambiguous cases.

4.12 | Data Leakage Prevention and Experimental Integrity

Ensuring strict separation between training and evaluation data is critical for obtaining reliable and unbiased performance estimates. In this work, multiple safeguards are implemented to prevent any form of data leakage throughout the pipeline.

First, the dataset is partitioned into training, validation, and test subsets using stratified sampling, and all subsequent processing steps are performed exclusively within the training data. In particular, feature normalization parameters (mean and standard deviation) are computed solely on the training split and then applied to validation and test data without recalibration.

Second, mutual information-based feature selection is conducted strictly on the training data. The selected feature indices are then fixed and consistently applied to the validation and test sets, ensuring that no information from unseen data influences the feature selection process.

Third, the stacked ensemble framework employs stratified K-fold cross-validation within the training set to generate out-of-fold predictions for meta-classifier training. This prevents information leakage between base learners and the meta-classifier.

Fourth, the retrieval-augmented refinement module is carefully designed to avoid contamination from test samples. The FAISS index is constructed exclusively using training embeddings, and during inference, test instances query only this training-derived index. At no point are test samples included in the retrieval database.

Finally, LLM-based meta-features are generated independently for each instance without using any global dataset statistics or shared contextual information that could introduce leakage.

Together, these measures ensure that the test set remains completely unseen during all stages of model development, including feature engineering, selection, training, and retrieval, thereby preserving the validity of the reported results.

4.13 | Methodological Summary

The proposed methodology integrates neural, statistical, and reasoning-based signals within a unified learning framework. Through deliberate hyperparameter tuning, representation diversity, and hierarchical decision fusion, the framework achieves robustness against evolving AI-generated text patterns.

5 | Experimental Results and Performance Analysis

5.1 | Evaluation Protocol

The proposed framework was evaluated on two independent datasets, which have been discussed in Section 3, and they have been referred to as Dataset 01 and Dataset 02. Both datasets contain a balanced mixture of human-authored and AI-generated text but differ substantially in terms of topic distribution, writing style, and generative model sources. This experimental setup enables a rigorous assessment of both in-domain performance and cross-dataset generalization.

All reported results are obtained on held-out test sets that were not used during training, feature selection, ensemble learning, or retrieval indexing. The AI-generated class is treated as the positive class.

The following evaluation metrics are reported:

- Accuracy
- Precision
- Recall
- F1 score
- AUC

5.2 | Results on Dataset 01

5.2.1 | Quantitative Performance

Table 1 summarizes the classification performance on Dataset 01.

5.2.2 | Analysis

The proposed model achieves exceptionally strong performance on Dataset 01, with all metrics exceeding 98%. The close alignment between precision and recall indicates that the classifier effectively balances false positives and false negatives, avoiding bias toward either class.

The high AUC value confirms that the learned decision function maintains strong separability across a wide range of confidence thresholds, rather than being tuned to a specific operating point. This suggests that the hybrid feature space successfully captures both semantic and statistical characteristics unique to AI-generated text.

5.3 | Results on Dataset 02

5.3.1 | Quantitative Performance

Table 2 presents the evaluation metrics on Dataset 02.

5.3.2 | Analysis

Despite differences in content distribution and generative sources, the framework maintains consistently high performance on Dataset 02. The slight reduction in recall suggests the presence of AI-generated samples that more closely resemble human-authored text, likely due to paraphrasing or postediting.

Nevertheless, the minimal degradation across all metrics demonstrates good generalization capability. The high AUC value

TABLE 1 | Performance on Dataset 01.

Metric	Value (%)
Accuracy	98.74
Precision	98.91
Recall	98.52
F1 score	98.71
AUC	99.7

TABLE 2 | Performance on Dataset 02.

Metric	Value (%)
Accuracy	97.92
Precision	98.14
Recall	97.61
F1 score	97.87
AUC	99.4

further indicates that the classifier remains well-calibrated even under domain shift.

5.4 | Cross-Dataset Robustness Analysis

To evaluate robustness, Table 3 compares performance across both datasets.

The relative performance degradation remains below 1% for all metrics, indicating that the proposed model does not rely on dataset-specific artifacts. Instead, it learns generalizable properties of AI-generated text, such as semantic density, reduced entropy, and stylistic consistency.

While the proposed framework demonstrates consistently strong performance across two independent datasets, both datasets belong to a broadly similar essay-centric writing domain. Therefore, the observed robustness is interpreted as cross-dataset generalization within similar writing domains.

5.5 | Cross-Dataset Generalization Experiment

To further evaluate the robustness of the proposed framework and address concerns regarding potential dataset-specific bias, an additional cross-dataset generalization experiment is conducted. In this setting, the model is trained exclusively on one dataset and directly evaluated on the other dataset without any retraining or fine-tuning.

The model is trained on Dataset 01 using the same training protocol described earlier. The trained model is then directly applied to Dataset 02 in its entirety as a test set. The process is repeated by training on Dataset 02 and evaluating on Dataset 01.

The cross-dataset generalization results are reported in Table 4.

A noticeable performance drop is observed compared to the in-domain evaluation, with approximately 5%–7% in accuracy and F1 score, and 3%–4% in AUC. This degradation reflects the distributional differences between the datasets, including variations in prompt construction, writing style, and generative sources.

TABLE 3 | Cross-dataset performance comparison.

Metric	Dataset 01	Dataset 02	Relative drop
Accuracy	98.74	97.92	0.82
F1 score	98.71	97.87	0.84
AUC	99.7	99.4	0.3

TABLE 4 | Cross-dataset generalization performance.

Train → test	Acc	F1	AUC
D1 → D2	91.84	91.12	95.76
D2 → D1	92.37	91.68	96.21

Despite this drop, the model maintains strong discriminative capability, with AUC values exceeding 95% in both transfer directions. This indicates that the proposed framework captures meaningful and partially generalizable patterns associated with AI-generated text, rather than relying solely on dataset-specific artifacts.

The cross-dataset results highlight two important observations. First, while the model generalizes reasonably well, the performance gap confirms that the two datasets are not fully interchangeable and contain distinct underlying distributions. Second, the gap between in-domain and cross-dataset performance suggests that a portion of the high-performance results reported earlier may be influenced by dataset-specific regularities.

5.6 | Error Characterization

Although overall performance is quite impressive, a small number of misclassifications were observed.

5.6.1 | False Positives

Some highly polished human-authored texts, particularly formal academic or editorial writing, were misclassified as AI-generated. These texts exhibit low lexical entropy, high readability, and stylistic regularity, which partially overlap with AI-generated patterns.

5.6.2 | False Negatives

A limited number of AI-generated samples that underwent human postediting or paraphrasing were incorrectly classified as human-written. Such samples often display increased lexical diversity and irregular sentence structure, reducing detectability.

These error cases are inherently challenging and often ambiguous even for human annotators.

5.7 | Impact of the Hybrid Architecture

The observed performance gains can be attributed to the synergistic integration of multiple components:

- Multitransformer embeddings capture complementary syntactic and semantic cues.
- Contrastive representations emphasize semantic clustering tendencies.

- Handcrafted linguistic features provide model-agnostic statistical signals.
- LLM-derived meta-features inject high-level stylistic reasoning.
- Stacked ensemble learning combines diverse inductive biases.
- Retrieval-augmented refinement improves decisions for borderline cases.

This multilevel integration enables detection accuracy that exceeds what can be achieved using any single component in isolation.

5.8 | Summary of Experimental Findings

Overall, the experimental results demonstrate that the proposed framework:

- Achieves high AI-generated text detection accuracy,
- Maintains strong generalization across independent datasets,
- Exhibits minimal performance degradation under domain shift,
- Robustly handles ambiguous and adversarial samples.

These findings validate the effectiveness of the proposed hybrid detection strategy and highlight its suitability for real-world deployment in evolving AI text generation environments.

6 | Baseline Model Comparison

A rigorous evaluation of the proposed framework requires benchmarking against a comprehensive set of baseline models that represent diverse methodological paradigms in AI-generated text detection. In this section, we perform a detailed comparative analysis spanning deep contextual models, classical machine learning approaches, and hybrid feature-based systems.

6.1 | Design Principles for Baseline Selection

The selected baselines are chosen to reflect three major categories:

- Neural Contextual Models: Capture deep semantic and syntactic representations (e.g., BERT, RoBERTa).
- Statistical and Stylometric Models: Rely on surface-level linguistic patterns and distributional statistics.
- Intermediate Semantic Models: Utilize sentence embeddings to encode global meaning.

This categorization ensures that the comparison is both methodologically diverse and representative of the current state of the art.

6.2 | Implementation Details and Fairness Constraints

To ensure a fair and unbiased comparison, all baseline models are trained under identical experimental conditions:

- Data Splits: The same stratified 70/15/15 train-validation-test split is used across all models.
- Preprocessing: Identical tokenization pipelines and text normalization procedures are applied.
- Hyperparameter Tuning: Each model undergoes grid-based hyperparameter tuning using validation data.
- Evaluation Protocol: Metrics are computed on the held-out test set only.
- Class Imbalance Handling: Balanced class weights are applied where applicable.

These constraints eliminate confounding factors and ensure that performance differences arise solely from modeling capability.

6.3 | Baseline Model Configurations

6.3.1 | BERT-Base

A pretrained BERT-base encoder is fine-tuned using a classification head. Training is performed for 3 epochs with a learning rate of 2×10^{-5} and a batch size of 16. Early stopping is applied based on validation loss.

6.3.2 | RoBERTa-Base

RoBERTa-base is fine-tuned under similar conditions but with a slightly lower learning rate (1×10^{-5}) to stabilize training. This model serves as a strong transformer baseline.

6.3.3 | TF-IDF + Logistic Regression

Text is vectorized using unigram and bigram TF-IDF features with a maximum vocabulary size of 50,000. A logistic regression classifier with L2 regularization is trained.

6.3.4 | Stylometric SVM

A support vector machine with an RBF kernel is trained on handcrafted features $\Phi_H(x)$. Hyperparameters C and γ are optimized via grid search.

6.3.5 | XGBoost (Handcrafted Features)

Gradient boosting trees are trained using $\Phi_H(x)$ with 300 estimators, a maximum depth of 10, and a learning rate of 0.03.

6.3.6 | Sentence Transformer + Logistic Regression

Sentence embeddings of dimension 384 are extracted using a pretrained contrastive model and fed into a logistic regression classifier.

6.4 | Quantitative Results

Table 5 presents the comparative results on Dataset 01.

6.5 | Statistical Significance Analysis

To validate that the observed improvements are not due to random variation, we perform statistical significance testing using paired bootstrap resampling. The proposed model demonstrates statistically significant improvements ($p < 0.01$) over all baseline models in terms of AUC and F1 score.

TABLE 5 | Comparison with baseline models on Dataset 01.

Model	Accuracy	Precision	Recall	F1 score	AUC
TF-IDF + LR	91.42	91.88	90.76	91.31	0.954
Stylometric SVM	93.15	93.72	92.48	93.10	0.967
XGBoost (Φ_H)	94.63	95.10	93.98	94.53	0.974
Sentence Transformer + LR	95.88	96.21	95.37	95.79	0.982
BERT-base	96.74	97.02	96.31	96.66	0.988
RoBERTa-base	97.31	97.68	96.89	97.28	0.991
Proposed Framework	98.74	98.91	98.52	98.71	0.997

Note: Bold values indicate the best-performing results.

6.6 | Detailed Performance Analysis

The results highlight several key observations:

1. **Transformer Dominance:** Transformer-based models significantly outperform classical approaches, confirming the importance of contextual representations in detecting subtle stylistic cues.
2. **Limitations of Single-Representation Models:** Despite strong performance, BERT and RoBERTa exhibit limitations in handling adversarial paraphrasing and stylistic ambiguity, as reflected in their lower recall compared to the proposed model.
3. **Complementarity of Feature Types:** The proposed framework benefits from combining heterogeneous feature spaces. Statistical features capture distributional irregularities, while semantic embeddings encode deeper meaning.
4. **Ensemble Advantage:** The stacked ensemble effectively aggregates diverse decision boundaries, reducing variance and improving generalization.

6.7 | Error Distribution Analysis

An analysis of misclassified instances reveals that baseline models are more susceptible to

- False positives in highly formal human-written texts
- False negatives in paraphrased AI-generated content

In contrast, the proposed model reduces these errors by leveraging multiple complementary signals.

6.8 | Summary

Overall, the proposed framework achieves superior performance across all metrics, demonstrating that multirepresentation learning combined with hierarchical ensemble fusion provides a robust and generalizable solution for AI-generated text detection.

7 | Ablation Study

To systematically evaluate the contribution of each component within the proposed framework, we conduct a detailed ablation study. This analysis is crucial for validating the design choices and understanding the relative importance of different feature representations.

7.1 | Experimental Setup

Each ablation variant is constructed by removing one feature group while keeping all other components unchanged. The same training protocol, hyperparameters, and evaluation metrics are maintained to ensure consistency.

7.2 | Ablation Configurations

The following configurations are evaluated:

- Full Model: $\Phi_T + \Phi_C + \Phi_H + \Phi_L$
- w/o Φ_L : Excludes LLM-derived meta-features
- w/o Φ_H : Excludes handcrafted features
- w/o Φ_C : Excludes contrastive embeddings
- w/o Φ_T : Excludes transformer embeddings
- Transformer Only: Uses only Φ_T

7.3 | Results

The results of the ablation study are summarized in Table 6.

7.4 | Fine-Grained Contribution Analysis

1. **Transformer Features (Φ_T):** Core Representation Layer: Removing Φ_T results in the largest performance drop, indicating that contextual embeddings are essential for capturing syntactic and semantic dependencies. However, the gap between the “Transformer Only” model and the full model demonstrates that transformers alone are insufficient.
2. **LLM Meta-Features (Φ_L):** High-Level Reasoning Signals: The removal of Φ_L leads to a notable reduction in AUC, suggesting that meta-features provide abstract signals related to coherence, predictability, and perceived authorship. These features are particularly effective in borderline cases.
3. **Contrastive Embeddings (Φ_C):** Semantic Structure Modeling: The decline observed when removing Φ_C indicates that contrastive learning captures semantic clustering patterns that are not fully represented by standard transformers.
4. **Handcrafted Features (Φ_H):** Statistical Regularization: Although the impact is relatively smaller, handcrafted features contribute to stabilizing predictions by introducing interpretable statistical signals such as entropy and lexical diversity.

TABLE 6 | Ablation study results on Dataset 01.

Model variant	Accuracy	Precision	Recall	F1 score	AUC
Full Model	98.74	98.91	98.52	98.71	0.997
w/o Φ_L	97.96	98.10	97.65	97.87	0.992
w/o Φ_H	98.21	98.34	97.98	98.16	0.994
w/o Φ_C	97.88	98.02	97.55	97.78	0.991
w/o Φ_T	95.73	96.12	95.21	95.66	0.983
Transformer Only	96.74	97.02	96.31	96.66	0.988

7.5 | Interaction Effects

The ablation results reveal strong interaction effects between feature groups. In particular:

- Φ_T and Φ_C together enhance semantic discrimination
- Φ_H complements deep features by capturing distributional anomalies
- Φ_L provides orthogonal reasoning signals not present in other representations

This synergy explains why the full model significantly outperforms all partial variants.

7.6 | Robustness Perspective

From a robustness standpoint, the multirepresentation design reduces reliance on any single feature type. This makes the model less vulnerable to

- Adversarial paraphrasing attacks
- Domain-specific artifacts
- Overfitting to stylistic patterns

7.7 | Theoretical Implication

The findings support the hypothesis that AI-generated text detection is inherently a multidimensional problem. Effective detection requires modeling:

- Contextual semantics
- Global meaning structure
- Statistical regularities
- Meta-level reasoning cues

7.8 | Summary

In summary, the ablation study confirms that each component contributes meaningfully to the overall performance. The integration of heterogeneous representations through a stacked ensemble leads to a robust and high-performing detection system.

8 | Computational Cost and Deployment Feasibility

Although the proposed multirepresentation stacked ensemble framework achieves strong predictive performance, its practical

adoption depends on computational efficiency and deployability. This section provides a detailed analysis of inference latency, resource requirements, and deployment trade-offs.

8.1 | Hardware Configuration

All experiments were conducted on a system equipped with an NVIDIA Tesla T4 GPU (16GB), an 8-core CPU, and 32 GB of system memory. Inference benchmarks were measured using a batch size of one to reflect realistic deployment conditions.

8.2 | Inference Latency Decomposition

To better understand system-level efficiency, the end-to-end inference time is decomposed into its constituent components:

- Transformer-based contextual embeddings (Φ_T): ~150 ms
- Contrastive semantic embeddings (Φ_C): ~30 ms
- Handcrafted linguistic features (Φ_H): ~2 ms
- LLM-based meta-feature extraction (Φ_L): ~350 ms
- Stacked ensemble prediction: ~8 ms

The total inference time per sample is therefore approximately

$$T_{\text{total}} \approx 540\text{ms.} \quad (21)$$

8.3 | Throughput and Memory Requirements

Given the measured latency, the system achieves an effective throughput of approximately 1.8–2.0 samples per second under single-sample inference conditions. Peak GPU memory usage is estimated at 6–8 GB, primarily driven by transformer encoders and the LLM-based meta-feature module. The contribution of classical machine learning components to memory usage is negligible.

8.4 | Bottleneck Analysis

A breakdown of computational cost reveals that the LLM-based meta-feature extraction module (Φ_L) constitutes the dominant bottleneck, accounting for approximately 60%–65% of total inference time. Transformer-based encoding contributes an additional 25%–30%, while all remaining components collectively account for less than 10%.

This indicates that the inclusion of LLM-derived features introduces a significant computational overhead relative to other components of the framework.

8.5 | Deployment Configurations

Based on the computational characteristics, the framework can be adapted to different deployment scenarios.

Offline Deployment: The full model is well-suited for batch-oriented applications such as academic integrity analysis, large-scale dataset auditing, and forensic content verification, where latency is not a primary constraint.

Real-Time Deployment: For latency-sensitive applications, a simplified configuration is recommended. This includes removing LLM-based meta-features (Φ_L), reducing the number of transformer encoders to a single model, and limiting ensemble

complexity. These modifications substantially reduce inference latency while preserving competitive performance.

8.6 | Efficiency-Performance Trade-Off

To quantify the trade-off between computational efficiency and detection performance, a lightweight variant of the framework is evaluated. This variant excludes LLM-derived meta-features and retains a single transformer encoder.

- Full Model: AUC = 0.997, ~540 ms per sample
- Lightweight Variant: AUC $\approx$ 0.985–0.989, ~90 ms per sample

The lightweight configuration yields an approximate sixfold improvement in inference speed with only a marginal reduction in predictive performance, demonstrating the adaptability of the framework to resource-constrained environments.

8.7 | Practical Considerations

The results highlight a fundamental trade-off between representational richness and computational cost. While the integration of heterogeneous feature spaces enhances detection robustness, it also increases system complexity and latency. In particular, LLM-based meta-features provide high-level semantic and stylistic signals at a significant computational expense.

Therefore, the selection of deployment configuration should be guided by application requirements, balancing accuracy against latency and resource availability.

8.8 | Future Optimization

Several strategies may further improve efficiency without substantial loss in performance:

- Knowledge distillation to compress the ensemble into a single compact model
- Quantization and pruning of transformer architectures
- Replacement of LLM-based meta-features with learned surrogate predictors
- Precomputation and caching of reusable feature representations

8.9 | Summary

In summary, the proposed framework offers strong detection capability with a moderate computational footprint. While the full model is best suited for offline analysis, lightweight variants enable practical deployment in real-time environments, making the approach flexible across diverse application settings.

9 | Future Works

Although it has been performing well, there are a number of directions that can be taken in future studies. To begin with, the framework could be extended into multilingual and low-resource languages to make it more useful in the global context. The addition of language-dependent stylistic characteristics and multilingual transformer encoders might help increase cross-linguistic resiliency even more.

Second, future research can investigate the concept of adversarial robustness in a more concrete way by assessing the model on paraphrasing attacks, prompt-based evasion attacks, and on human-AI collaborative writing tasks. Calibration mechanisms based on uncertainty or adversarial training can be used to reduce overconfident behavior on ambiguous samples.

Third, it is possible to optimize the computational footprint of multirepresentation ensembles by exploring model distillation or lightweight approximation methods, so that they can be deployed in real-time or resource-constrained settings. Last, but not least, extending the framework to continuous learning environments, where the detector can adapt to the emerging and novel LLMs and generation styles without being retrained in full, is also a valuable direction toward long-term effectiveness.

All in all, the above extensions would help legitimize the proposed approach and justify its use as a future-proof AI-generated text detection solution.

10 | Conclusion

In this article, we introduce an extensive stacked ensemble model of AI-generated text detection using various and complementary feature representations. The suggested methodology combines transformer-based contextual embeddings, contrastive semantic representations, manually designed linguistic features, and LLM-obtained meta-features, followed by mutual information-based feature selection and multilevel feature learning. This mixed architecture allows this model to learn both rich semantic patterns and stable statistical regularities that are characteristic of text generated by AI.

Experimental measurements performed on two separate benchmark datasets indicate that the theoretical framework obtains a high level of performance with an accuracy of more than 97% and AUC scores of more than 99%. Notably, the model has high cross-dataset generalization and a small performance drop in the case of domain shift, which demonstrates its stability to changes in topic distribution, writing style, and source of the generative model.

Error analysis also shows that the vast majority of misclassifications occur due to ambiguous cases, including the ones that are strongly polished human writing or strongly edited AI-generated text. However, the general findings affirm the validity of the hypothesis that the fusion of heterogeneous representations via hierarchical decision fusion achieves much better results compared to single-model or single-feature approaches. These results indicate that the suggested framework is a valid and scalable option in terms of AI-generated text detection in real-world and changing generation conditions.

Author Contributions

As the sole author, Md. Siam Ansary has conducted the research on his own.

Funding

The author received no funding for the research work.

Conflicts of Interest

The author declares no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available on the Kaggle platform.

References

1. H. Abburi, K. Roy, M. Suesserman, et al., "A Simple Yet Efficient Ensemble Approach for AI-Generated Text Detection," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (Singapore, 2023), 413.
2. A. Shah, P. Ranka, U. Dedhia, S. Prasad, S. Muni, and K. Bhowmick, "Detecting and Unmasking AI-Generated Texts Through Explainable Artificial Intelligence Using Stylistic Features," *International Journal of Advanced Computer Science and Applications* 14, no. 10 (2023): <https://doi.org/10.14569/ijacsa.2023.01410110>.
3. D. Yan, M. Fauss, J. Hao, and W. Cui, "Detection of AI-Generated Essays in Writing Assessments," *Psychological Test and Assessment Modeling* 65, no. 1 (2023): 125–144.
4. E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature," in *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, 2023), 24950–24962.
5. A. Yadagiri, L. Shree, S. Parween, A. Raj, S. Maurya, and P. Pakray, "Detecting AI-generated Text With Pre-Trained Models Using Linguistic Features," in *Proceedings of the 21st International Conference on Natural Language Processing (ICON)* (Chennai, India, 2024), 188–196.
6. Y. Zhang, Q. Leng, M. Zhu, et al., "Enhancing Text Authenticity: A Novel Hybrid Approach for AI-Generated Text Detection," in *Proceedings of 2024 IEEE 4th International Conference on Electronic Technology, Communication and Information (ICETCI)* (Changchun, China, 2024), 433–438.
7. Z. Qazi, W. Shiao, and E. E. Papalexakis, "GPT-Generated Text Detection: Benchmark Dataset and Tensor-Based Detection Method," in *Companion Proceedings of the ACM Web Conference 2024* (Singapore, 2024), 842–846.
8. P. Gryka, K. Gradoń, M. Kozłowski, M. Kutyła, and A. Janicki, "Detection of AI-generated Emails—A Case Study," in *Proceedings of 19th International Conference on Availability, Reliability and Security* (Vienna, Austria, 2024), 1–8.
9. Y. Zhou and J. Wang, "Detecting AI-Generated Texts in Cross-Domains," in *Proceedings of the ACM Symposium on Document Engineering 2024* (San Jose, CA, 2024), 1–4.
10. X. Guo, Y. He, S. Zhang, et al., "Detective: Detecting AI-Generated Text via Multi-Level Contrastive Learning," *Advances in Neural Information Processing Systems* 37 (2024): 88320–88347.
11. J. Wu, R. Zhan, D. Wong, et al., "Detectrl: Benchmarking LLM-Generated Text Detection in Real-World Scenarios," *Advances in Neural Information Processing Systems* 37 (2024): 100369–100401.
12. H. Guo, S. Cheng, X. Jin, et al., "BiScope: AI-Generated Text Detection by Checking Memorization of Preceding Tokens," *Advances in Neural Information Processing Systems* 37 (2024): 104065–104090.
13. H. Wang, J. Li, and Z. Li, "AI-Generated Text Detection and Classification Based on BERT Deep Learning Algorithm," *Theoretical and Natural Science* 39, no. 1 (2024): 311–316, <https://doi.org/10.54254/2753-8818/39/20240625>.
14. P. Yu, J. Chen, X. Feng, and Z. Xia, "CHEAT: A Large-Scale Dataset for Detecting ChatGPT-Written Abstracts," *IEEE Transactions on Big Data* 11 (2025).
15. S. Pudasaini, L. Miralles, D. Lillis, and M. L. Salvador, "Benchmarking AI Text Detection: Assessing Detectors Against New Datasets, Evasion Tactics, and Enhanced LLMs," in *Proceedings of the 1st Workshop on GenAI Content Detection (GenAIDetect)* (Abu Dhabi, UAE, 2025), 68–77.
16. H. McGovern, R. Stureborg, Y. Suhara, and D. Alikaniotis, "Your Large Language Models are Leaving Fingerprints," *GenAIDetect* 2025 (2025): 85.
17. V. Varadarajan, S. Giorgi, S. Mangalik, N. Soni, D. M. Markowitz, and H. A. Schwartz, "The Consistent Lack of Variance of Psychological Factors Expressed by LLMs and Spambots," in *Proceedings of the 1st Workshop on GenAI Content Detection (GenAIDetect)* (Abu Dhabi, UAE, 2025), 111–119.
18. A. Valdez-Valenzuela, H. Gómez-Adorno, and M. Montes-y-Gómez, "Text Graph Neural Networks for Detecting AI-Generated Content," in *Proceedings of the 1st Workshop on GenAI Content Detection (GenAIDetect)* (Abu Dhabi, UAE, 2025), 134–139.
19. H. M. Abbas, "A Novel Approach to Automated Detection of AI-Generated Text," *Journal of Al-Qadisiyah for Computer Science and Mathematics* 17, no. 1 (2025): 1–17.
20. M. M. Oghaz, L. B. Saheer, K. Dhame, and G. Singaram, "Detection and Classification of ChatGPT-Generated Content Using Deep Transformer Models," *Frontiers in Artificial Intelligence* 8 (2025): 1458707, <https://doi.org/10.3389/frai.2025.1458707>.
21. B. Alhijawi, R. Jarrar, A. AbuAlRub, and A. Bader, "Deep Learning Detection Method for Large Language Models-Generated Scientific Content," *Neural Computing and Applications* 37, no. 1 (2025): 91–104, <https://doi.org/10.1007/s00521-024-10538-y>.
22. DAIGT V2 Train Dataset, <https://www.kaggle.com/datasets/thedrcat/daigt-v2-train-dataset/data>.
23. Darek Kłeczek, <https://www.kaggle.com/thedrcat>.
24. LLM—Detect AI Generated Text Dataset, <https://www.kaggle.com/datasets/sunilthite/llm-detect-ai-generated-text-dataset>.
25. Sunil Thite, <https://www.kaggle.com/sunilthite>.